

MEV Research: Literature Review

👤 Person in charge	👤 Chen Xuan
☰ Contributors	/
📅 Date	@October 17, 2024
⚙️ Status	In progress

- *This post aims to provide an overview of MEV research in the blockchain area and to address philosophical issues from an academic perspective, which involves limited practical implementations.*
- *When discussing MEV, it is important to consider how the perspectives and incentives of market participants involved in MEV supply chain. This post focuses on the ecosystem and how to minimize the negative ecological impact of MEVs.*
- *The MEV concept is not important since the MEV activities are evolving constantly. If you are unfamiliar with it, please refer to this definition, which is given in a literature: **The ability to set the order of transactions before they are confirmed within a block introduces different types of profit seeking behaviors and strategies that are often referred to as MEV.***

The Core Question

Is MEV good or bad?

- What is MEV? (**Definition**)
 - Modeling
 - Taxonomy
- How to find MEV? (**Detection**)

- with Event Matching (Traditional Method)
- with Machine Learning Method
- Is MEV good or bad? (**Judgement**)
 - Estimation
 - Legitimacy
- How should we treat MEV? (**Mitigation**)
 - From Academic perspective
 - From Industry perspective (ethresear.ch)

What is MEV

Modeling

- **Cyclic Arbitrage in Decentralized Exchanges**
- <https://dl.acm.org/doi/abs/10.1145/3487553.3524201>
- Cyclic Arbitrage Model

Arbitrage model: A cyclic transaction between n tokens $A_1, A_2, \dots, A_n$ is a sequence of n trades:

Trade 1: Exchange δ_1 of A_1 with δ_2 of A_2 .

Trade 2: Exchange δ_2 of A_2 with δ_3 of A_3 .

...

Trade n : Exchange δ_n of A_n with δ'_1 of A_1 .

The Cyclic Arbitrage Model

THEOREM 1. *For a given cycle $A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n \rightarrow A_1$ with n tokens, there exists an arbitrage opportunity for the cyclic transaction if the product of exchange rates $\frac{a_{2,1} \cdot a_{3,2} \cdot \dots \cdot a_{1,n}}{a_{1,2} \cdot a_{2,3} \cdot \dots \cdot a_{n,1}} > \frac{1}{r_1^n \cdot r_2^n}$, where $a_{i,j}$ denotes the liquidity of token A_i in the liquidity pool with token A_j . Meanwhile, the arbitrage cannot benefit from the reversed direction $A_1 \rightarrow A_n \rightarrow \dots \rightarrow A_2 \rightarrow A_1$ for cyclic transactions.*

The condition when an arbitrage opportunity is profitable

Algorithm 1 Compute the equivalent liquidity of the cycle

```

1:  $a'_{1,n} \leftarrow a_{1,2}$ 
2:  $a'_{n,1} \leftarrow a_{2,1}$ 
3: for  $i$  from 2 to  $n - 1$  do
4:    $a'_{1,n} \leftarrow \frac{a'_{1,n} \cdot a_{i,i+1}}{a_{i,i+1} + r_1 \cdot r_2 \cdot a'_{n,1}}$ 
5:    $a'_{n,1} \leftarrow \frac{r_1 \cdot r_2 \cdot a'_{n,1} \cdot a_{i+1,i}}{a_{i,i+1} + r_1 \cdot r_2 \cdot a'_{n,1}}$ 
6: end for

```

The optimal trading volume of a cyclic arbitrage

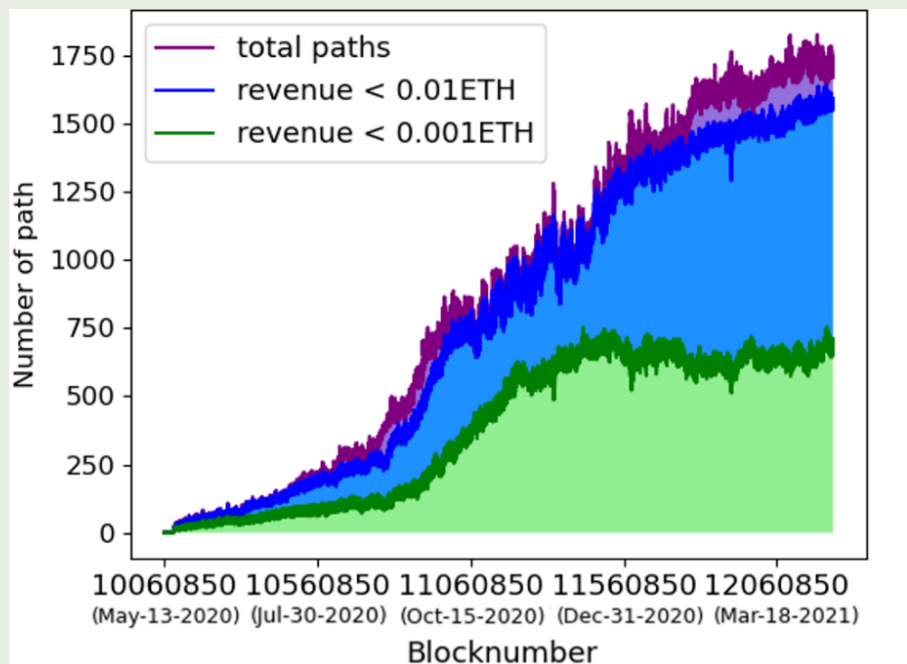


Figure 2: Number of exploitable opportunities in Uniswap V2 over time. The purple line represents the number of cycles that provide revenue higher than 0.0001 ETH. The green represents the number of cycles whose revenue is under 0.001 ETH. The blue line represents the number of cycles whose revenue is under 0.01 ETH.

- **Impact and User Perception of Sandwich Attacks in the DeFi Ecosystem**
- <https://dl.acm.org/doi/abs/10.1145/3491102.3517585>

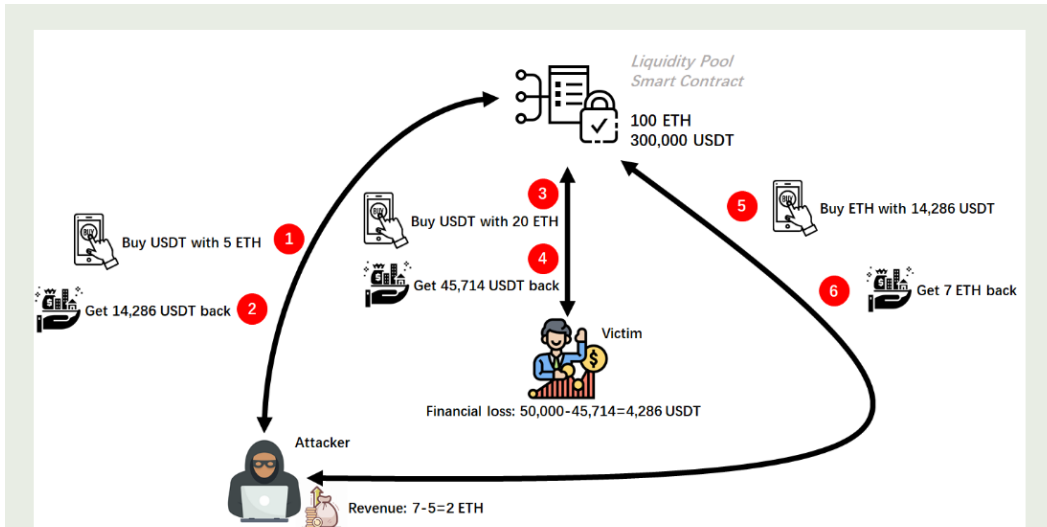


Figure 2: An example of sandwich attacks on AMM DEXes with constant product function and no commission fee. The attacker front-runs the victim transaction with a buy order of USDT and back-runs the victim transaction with a sell order. The reserves in the liquidity pool change after each transaction. The original state is 100 ETH/300,000 USDT. Then it becomes 105 ETH/285,714 USDT after the first transaction from the attacker. After the victim transaction, the reserves in the liquidity pool change to 125 ETH/240,000 USDT. Finally, the reserves change to 118 ETH/254,286 USDT after the second transaction from the attacker.

Sandwich Attack Mechanism

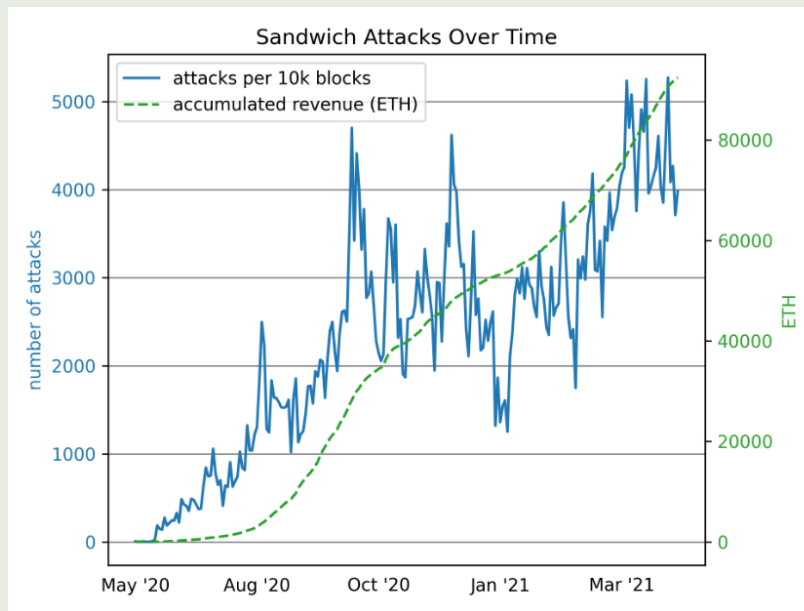
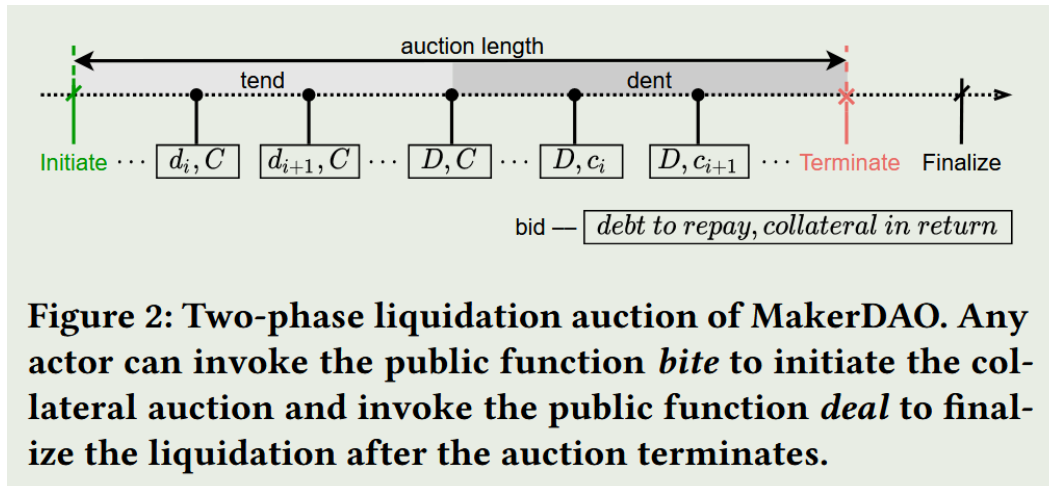


Figure 3: The number of sandwich attacks we observed and the profit attackers accumulate (the financial loss of victims) over time.

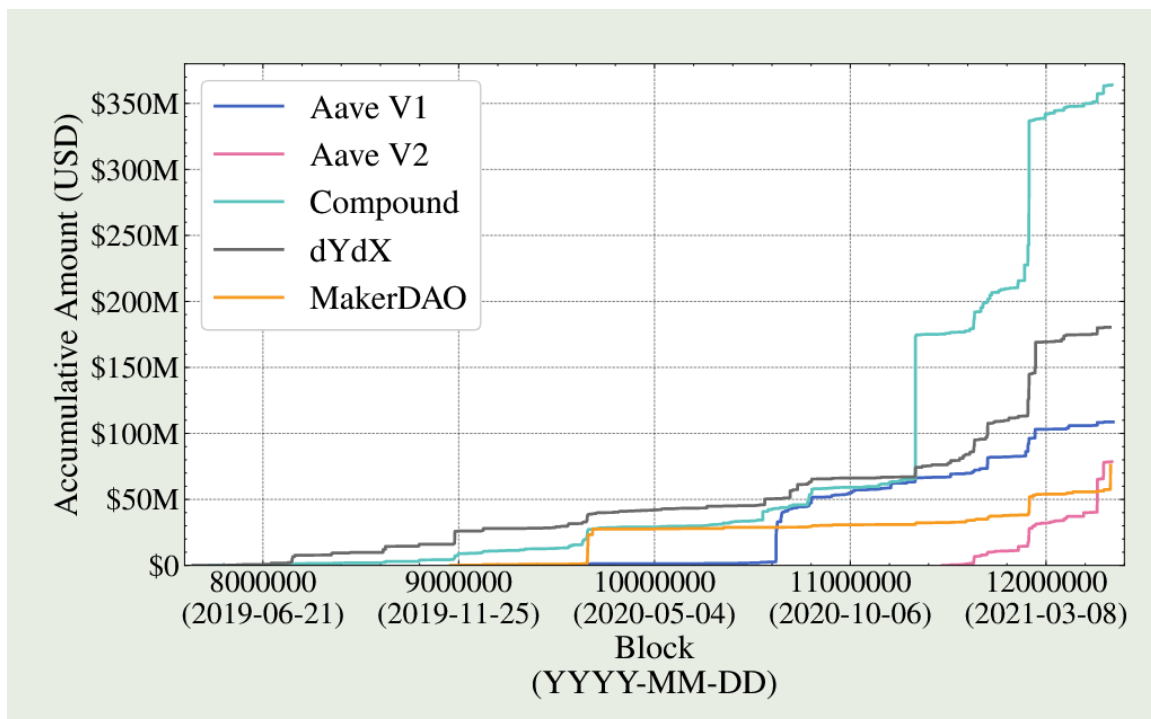
Impact of Sandwich Attack

- **An empirical study of DeFi liquidations: incentives, risks, and instabilities**
- <https://dl.acm.org/doi/abs/10.1145/3487552.3487811>
- Liquidation Model and it's risks



Auction liquidation model

- In the tend phase, liquidators compete by bidding to repay parts of the debt in exchange for the entire collateral.
- In the dent phase, liquidators compete by bidding to accept decreasing amounts of collateral in exchange for the full debt (i.e., D) they will end up repaying

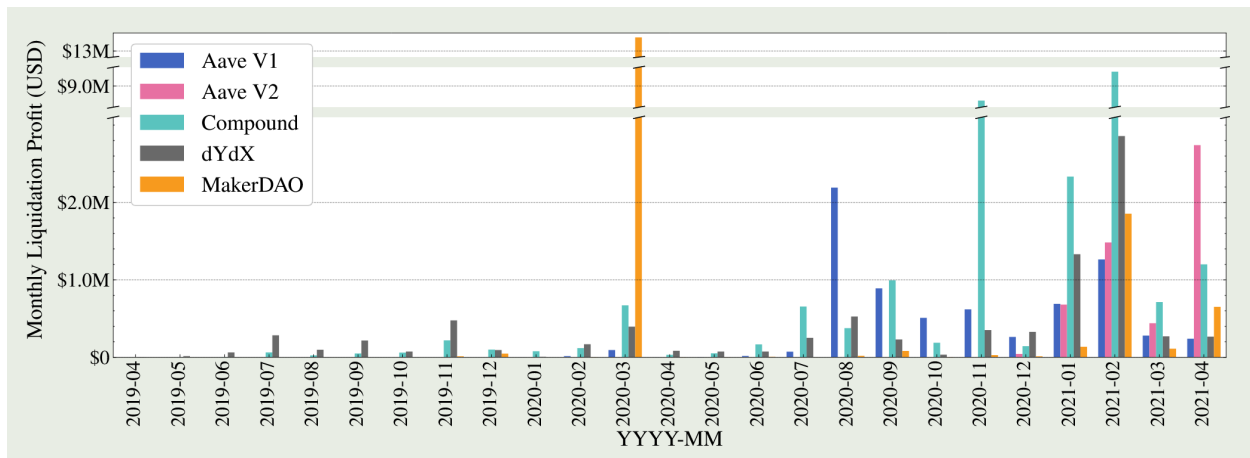


The accumulative collateral sold through liquidation (April, 2019 to April, 2021).

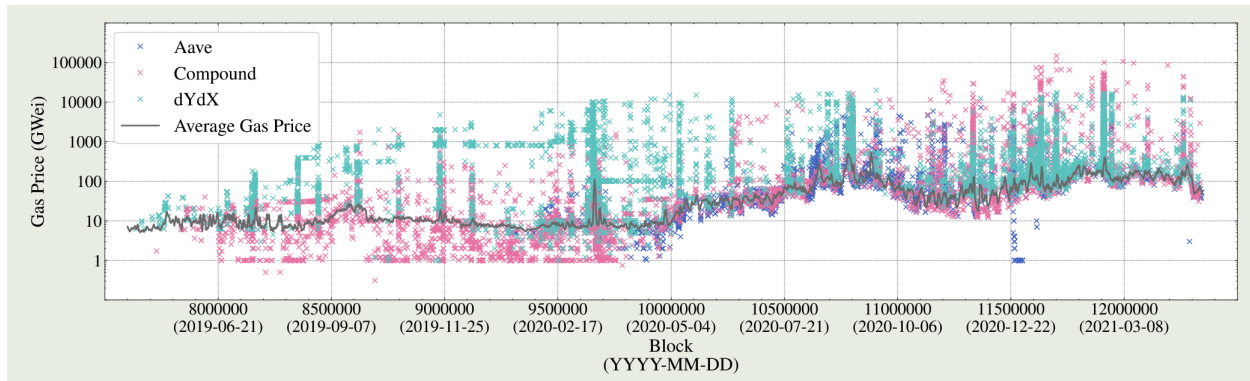
Table 1: Number of the liquidations and liquidators on Aave, Compound, dYdX, MakerDAO and their average profit. We measure the number of liquidators based on their unique Ethereum address. We notice that some liquidators operate on multiple lending markets.

Platform	Liquidations	Liquidators	Average Profit
Aave V1	3,809	665	10.76K USD
Aave V2	1,039	125	43.12K USD
Compound	6,766	657	39.94K USD
dYdX	9,762	600	14.30K USD
MakerDAO	6,762	140	115.84K USD
Total	28,138	2,011	31.62K USD

Number of liquidators and average profit



Monthly accumulated liquidator profit



Gas prices paid by liquidators

An unreasonable parameter settings (i.e., the close factor) will lead to some risks

- The Problem of **Over-Liquidation**
- **Bad Debts:** We define a borrowing position as a bad debt if it is financially rationale for neither the borrowers nor the lending platform to close the position.

- **Price of MEV: Towards a Game Theoretical Approach to MEV**
- <https://dl.acm.org/doi/abs/10.1145/3560832.3563433>
- Modeling MEV with Game Theory

Definition 3.1. Given a set of players $\mathcal{P}$, a random positive variable $\mathcal{B}$, a gossip network graph G and an auction mechanism, $\mathbb{A} = (\mathbf{x}, \mathbf{pr})$, an MEV game is $\mathbb{G} = (\mathcal{P}, G, \mathcal{B}, \mathbb{A}, (c_i)_{i=1,..,n})$. We say the game is symmetric if all players share the same features. That is, $V_i = V_j, c_i = c_j$ for all i, j , $\mathbb{A}$ are symmetric functions and G is a homogeneous graph.

Modeling the MEV Game

Definition 3.4. Let $\mathbb{G}$ be an MEV stage game, we say that a tuple of strategies $(S_1, \dots, S_n)$ is a *Nash equilibrium* (NE) if for each player P_i

$$u_i(S_i, S_{-i}) \geq u_i(\tilde{S}_i, S_{-i}), \text{ for all strategies } \tilde{S}_i.$$

In other words, if players are taking the strategies $(S_1, \dots, S_n)$, none of them have incentives to deviate unilaterally. We denote $\text{NE}(\mathbb{G})$ the set of all Nash equilibrium.

The MEV Game can reach Nash equilibrium

- The notion of **Nash Equilibrium** is not strong in permissionless environments
- The set of Nash Equilibrium are not **Sybil Resistant**

- **Clockwork Finance: Automated Analysis of Economic Security in Smart Contracts**

- <https://arxiv.org/abs/2109.04347>

- Need to consider the probability of appending multiple blocks in **Proof-of-Work**

Definition 3 (Weighted MEV).

$$\text{WMEV}(P, s) = \sum_{k=1}^{\infty} p_k \cdot k\text{-MEV}(P, s)$$

As a simple example, consider a miner P who controls a fraction f of the total hash power. If we assume that mining is modeled as a random oracle and that there is no selfish mining, then the probability p_k that P mines exactly k consecutive blocks is $p_k = f^k(1 - f)$. Suppose further that the extra MEV obtained per extra mined block is a constant m . For this simplified example, we can compute the WMEV as:

$$\text{WMEV}(P, s) = \sum_{k=1}^{\infty} f^k(1 - f)(km) = \frac{fm}{(1 - f)}$$

Equipped with this, we can also generalize the definition of Defi composability to include WMEV. For this, MEV in Definition 1 will be replaced by WMEV.

Weighted MEV Definition

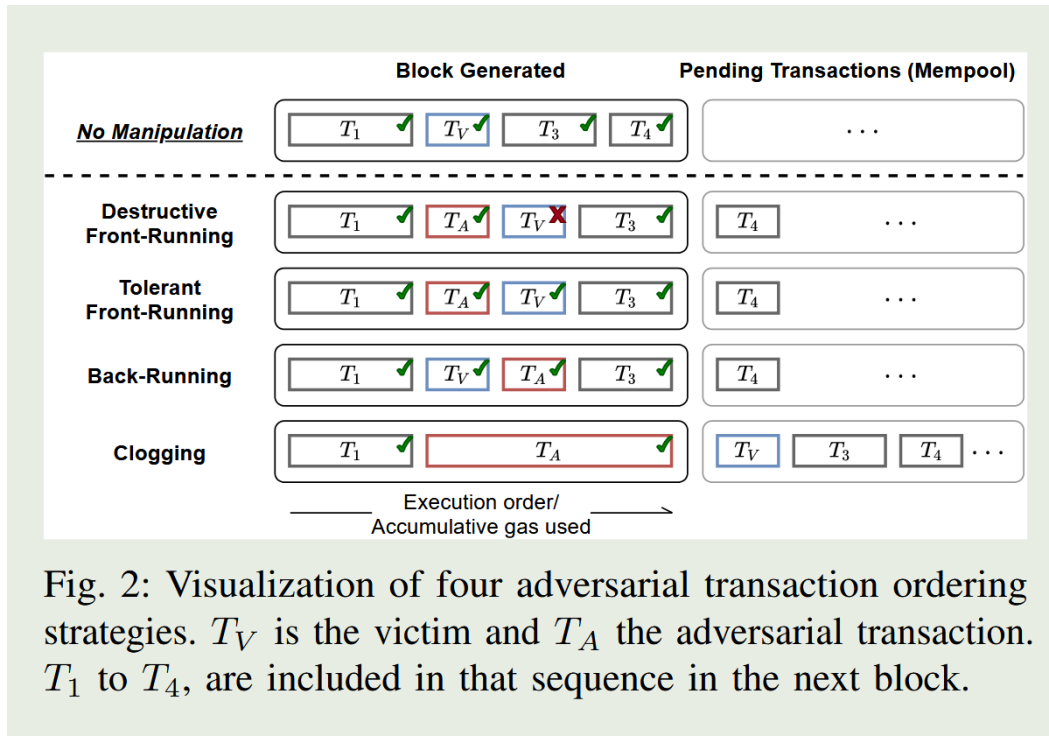
-
- **WIP: Are Builders Using Multi-block MEV (MMEV) Opportunities?**

- <https://arxiv.org/abs/2303.04430>
- Under Proof-of-Stake, the method of MMEV including:
 - **collusive MMEV**: the pool operator of any sequence of proposers elected for **k-consecutive slots** colludes with a builder by selling consecutive block space
 - **non-collusive MMEV**: builders can rely on elected proposers to accept a targeted bidding strategy through a relay
- Strategies including:
 - **Discrete strategies**: seek to reach a specific set of pre-defined threshold values provoking an on-chain scenario such as the liquidation of a vulnerable position.
 - **Continuous strategies**: seeking to structure transaction flow throughout the duration of the consecutive slots, by selecting and withholding transactions from the mempool.

Taxonomy

Notice: The continuously changing landscape of MEV has resulted in a lack of consensus or agreed-upon taxonomy within industry and academia on how to categorize different types of MEV.

-
- **Quantifying Blockchain Extractable Value: How dark is the forest?** (IEEE SP 2022, CCF A)
 - <https://arxiv.org/abs/2101.05511>
 - Transaction Ordering Taxonomy



Taxonomy of MEVs

- Destructive Front-Running
 - Tolerating Front-Running
 - Back-Running
 - Clogging/Suppression
-
- **A Study of MEV Extraction Techniques on a First-Come-First-Served Blockchain**
 - <https://arxiv.org/abs/2308.06513>
 - Algorand, BFT+PPoS (Private Blockchain)

Table 1: Transaction Ordering Techniques on Algorand

Ordering Technique	Block State	Network State
Destructive Frontrunning	✓	✗
Tolerating Frontrunning	✗	✗
Backrunning	✓	✓
Clogging/Suppresion	✓	✓

- Different from public blockchain network, a **FCFS blockchain** cannot change **network state** and enforce frontrunning victim transaction, but proposers/operators can execute such attacks since they control transaction release order (**block state**) to the network.
- *Tolerating Frontrunning* is not applicable as guaranteeing a transaction's execution would require observing that transaction in the network first. But *Destructive Frontrunning* can be performed.
- *Backrunning* and *Clogging/Suppresion* are executable.

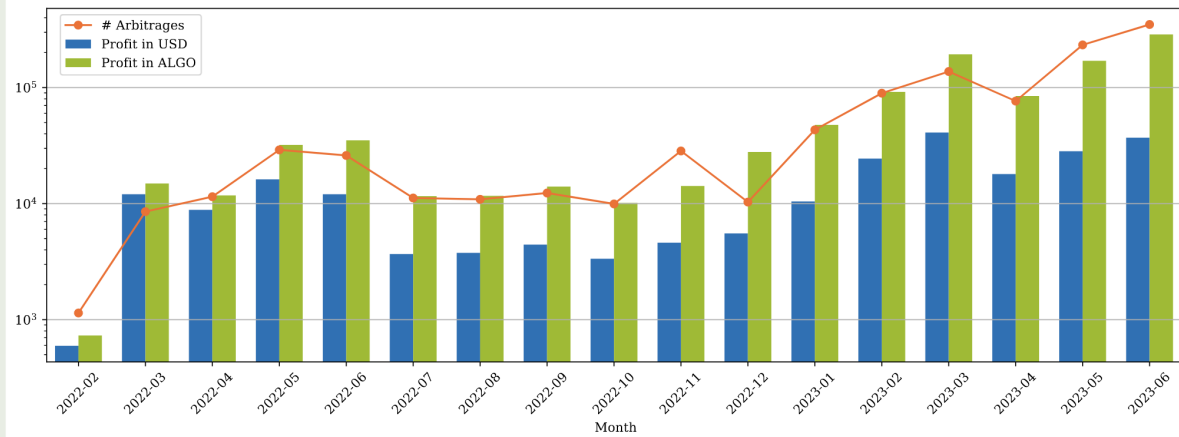


Figure 3: Timeline of the number of arbitrages (orange plot), profits in USD (blue bars), and profits in ALGO (green bars) from February 2022 to June 2023.

Arbitrage Profits on Algorand

- **SoK: Transparent Dishonesty: Front-Running Attacks on Blockchain**

- https://link.springer.com/chapter/10.1007/978-3-030-43725-1_13
 - SoK = Systemization of Knowledge
 - Taxonomy of Front-running
 - **displacement**: A trying to submit a bid in an auction, B copying it and submit first.
 - **insertion**: Insert and make a profit from the price difference
 - **suppression**: Suppress the block space to prevent some transaction go into effect with front-running
 - Each have two variants: **asymmetric** and **bulk**. asymmetric means transactions are performing different operations (**asymmetric displacement**). bulk means they are competing with same operation.
-

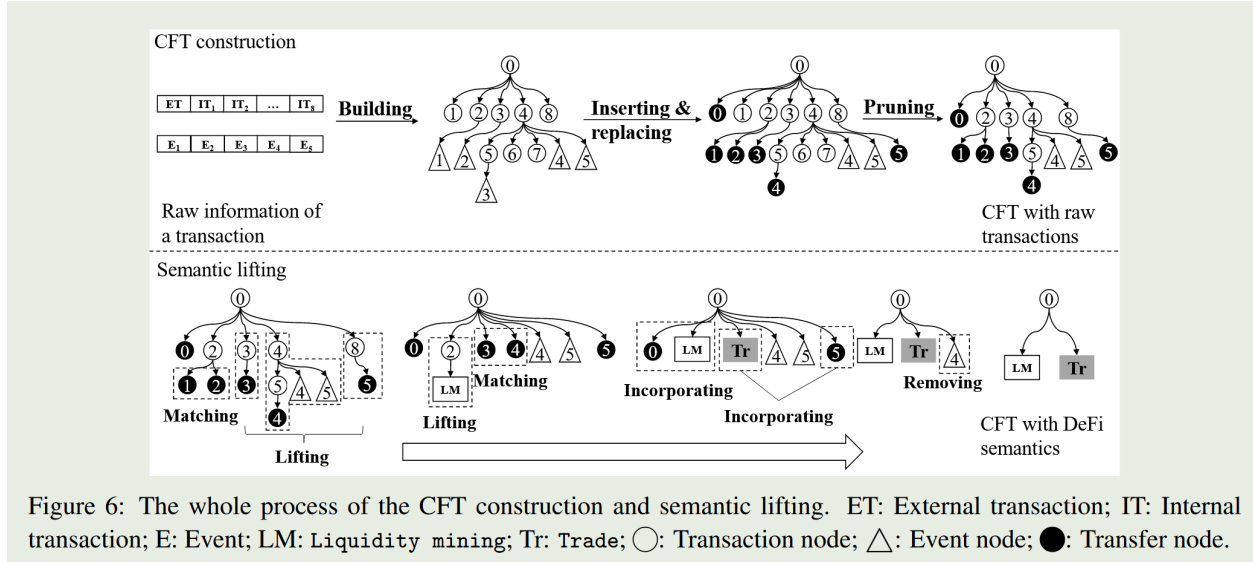
How to Find MEV

- The main method of detecting MEV transactions by gathering transfer information and specific events, especially semantic information in transactions

DeFi Actions

- **DeFiRanger: Detecting Price Manipulation Attacks on DeFi Applications**
- <https://arxiv.org/abs/2104.15068>
- Challenges:
 - DeFi apps often consist of multiple smart contracts that interact with each other, thus a MEV transaction may include **huge amount of internal transactions**.
 - Semantic gap between raw transactions (`from` , `to` , `calldata`) and high level DeFi Semantics (*xx address trade xx for xx using xx protocol*)
- Method to fill semantic gap including:
 - Raw Semantics → **DeFi Actions** → MEV Activities
 - Machine Learning

- This paper solve the first challenge by **pruning unnecessary transactions** (constructing **Cash Flow Tree**), and solve the second challenge with a **Sematic-Lifting** method



- CFT Construction
 - Construct raw transaction into a tree with Transaction node / Event node / Transfer node
 - Removing internal transaction that is not related with the following seven DeFi actions.

Table 1: The definition of DeFi actions. F_f means the value of the field f inside an (external or internal) transaction or an event that records a transfer of an ERC20 token.

DeFi Action	Symbol	Attributes	Condition	Assignment	Description
Basic actions	Ba				Basic actions include the following three types of transfer : T , T_m , and T_b .
Transfer(normal)	T	$spender$, $recipient$, $amount$, $asset$	$F_{spender} \neq F_{recipient}$ and $F_{spender} \neq \text{zeroaddress}$ and $F_{recipient} \neq \text{zeroaddress}$ and $F_{amount} > 0$	$spender = F_{spender}$; $recipient = F_{recipient}$; $asset = F_{asset}$; $amount = F_{amount}$	An account ($spender$) transfers a certain amount ($amount$) of certain asset ($asset$) to another account ($recipient$).
Transfer(minting)	T_m	$spender$, $recipient$, $amount$, $asset$	$F_{spender} == \text{zeroaddress}$; and $F_{amount} > 0$;	$spender = F_{spender}$; $recipient = F_{recipient}$; $asset = F_{asset}$; $amount = F_{amount}$	An ERC20 token contract mints a certain amount ($amount$) of certain asset ($asset$) to an account ($recipient$).
Transfer(burning)	T_b	$spender$, $recipient$, $amount$, $asset$	$F_{recipient} == \text{zeroaddress}$ and $F_{amount} > 0$	$spender = F_{spender}$; $recipient = F_{recipient}$; $asset = F_{asset}$; $amount = F_{amount}$	An account ($spender$) burns a certain amount ($amount$) of certain asset ($asset$).
Advanced actions	Aa				Advanced actions include the below three ones: LM , LC , and Tr .
Liquidity Mining	LM	$operator$, $recipient$, $pool$, $asset.in$, $asset.out$, $amount.in$, $amount.out$	Exists T and T_m and $T.asset \neq T_m.asset$	$operator = T.spender$; $recipient = T_m.recipient$; $pool = T.recipient$; $asset.in = T.asset$; $asset.out = T_m.asset$; $amount.in = T.amount$; $amount.out = T_m.amount$	An account ($operator$) of a liquidity provider deposits certain amount ($amount.in$) of certain asset ($asset.in$) to provide liquidity in a liquidity pool ($pool$) of a DeFi app, which then mints certain amount ($amount.out$) of its LP token ($asset.out$) to an account ($recipient$) that is specified by the liquidity provider.
Liquidity Cancel	LC	$operator$, $recipient$, $pool$, $asset.in$, $asset.out$, $amount.in$, $amount.out$	Exists T and T_b and $T.asset \neq T_b.asset$	$operator = T_b.spender$; $recipient = T.recipient$; $pool = T.spender$; $asset.in = T_b.asset$; $asset.out = T.asset$; $amount.in = T_b.amount$; $amount.out = T.amount$	An account ($operator$) of a liquidity provider burns a certain amount ($amount.in$) of certain DeFi app's LP token ($asset.in$) to redeem deposits, a pool ($pool$) of the app then transfers a certain amount ($amount.out$) of certain asset ($asset.out$) to an account ($recipient$) that is specified by the liquidity provider.
Trade	Tr	$operator$, $recipient$, $pool$, $asset.in$, $asset.out$, $amount.in$, $amount.out$	(Exists two $transfers(normal)$ $T1$ and $T2$ and $T1.asset \neq T2.asset$ and $T1.recipient == T2.spender$) or (exists LM and LC and $LM.recipient == LC.operator$ and $LM.asset.in \neq LC.asset.out$ and $LM.asset.out \neq LC.asset.in$)	$operator = T1.spender$ or ($LM.operator$); $recipient = T2.recipient$ or ($LC.recipient$); $pool = T1.recipient$ or ($LM.recipient$); $asset.in = T1.asset$ or ($LM.asset.in$); $asset.out = T2.asset$ or ($LC.asset.out$); $amount.in = T1.amount$ or ($LM.amount.in$); $amount.out = T2.amount$ or ($LC.amount.out$)	An account ($operator$) of a trader sells a certain amount ($amount.in$) of certain asset ($asset.in$) for a certain amount ($amount.out$) of certain asset ($asset.out$) in a liquidity pool ($pool$) of an AMM, and the trader specifies an account ($recipient$) as recipient.

DeFi Actions defined in this paper

- **Semantic-Lifting**

- Repeatedly merges child nodes in pairs in CFT until they cannot be merged.

Table 2: Patterns of DeFi actions to detect price manipulation attacks.

	Combination of DeFi actions	Detection rules	Description
Direct price manipulation attack	$Tr1 \rightarrow^a Ba \text{ (or } Tr3) \rightarrow Tr2$	Exists a sequence of DeFi actions: $Tr1$, Ba (or $Tr3$), and $Tr2$ and $Tr1.operator == Tr1.recipient$ and $Tr2.operator == Tr2.recipient$ and $Tr1.operator == Tr2.operator$ and $Tr1.pool == Tr2.pool$ and $Tr1.asset.in == Tr2.asset.out$ and $Tr1.asset.out == Tr2.asset.in$ and $Tr1.amount.out == Tr2.amount.in$ and $Tr1.amount.in < Tr2.amount.out$ and $((Ba.spender! = Tr1.operator$ and $Ba.recipient == Tr1.pool$ and $Ba.asset! = Tr1.asset.out)$ or $(Tr3.operator! = Tr1.operator$ and $Tr3.pool == Tr1.pool$ and $Tr3.asset.out == Tr1.asset.out))$	An attacker ($Tr1.operator$) inflates an asset's ($Tr1.asset.out$) price in a liquidity pool ($Tr1.pool$) of an AMM via a trade ($Tr1$) and then abuses the exposed interface of a vulnerable app ($Ba.spender$ or $Tr3.operator$) to further inflate the asset's ($Tr1.asset.out$) price in the pool on behalf of the vulnerable app. Finally, the attacker launches a reverse trade ($Tr2$) in the same pool ($Tr1.pool$) to gain profits. Above three steps are consistent to the three steps in Section 3.2, but we extend the middle step to transfer and trade for detecting more attacks.
Indirect price manipulation attack	$Tr1 \rightarrow Ba \text{ (or } Aa) \rightarrow Tr2$	Exists a sequence of DeFi actions: $Tr1$, Ba (or Aa), and $Tr2$ and $Tr1.operator == Tr1.recipient$ and $Tr2.operator == Tr2.recipient$ and $Tr1.operator == Tr2.operator$ and $Tr1.pool == Tr2.pool$ and $Tr1.asset.in == Tr2.asset.out$ and $Tr1.asset.out == Tr2.asset.in$ and $Tr1.amount.out == Tr2.amount.in$ and $Tr1.amount.in ==^b Tr2.amount.out$ and $(Ba.recipient == Tr1.operator$ or $(Aa.pool! = Tr1.pool$ and $Aa.recipient == Tr1.operator))$	An attacker ($Tr1.operator$) leverages a trade ($Tr1$) to indirectly manipulate the price in a vulnerable app ($Ba.spender$ or $Aa.pool$) and then profits from the vulnerable DeFi app via certain DeFi action (Ba or Aa). Finally, the attacker launches a reverse trade ($Tr2$) to redeem costs. Similarly, above three steps are consistent to the three steps in Section 3.3, but we extend the middle step to any DeFi action that can profit the attacker for detecting more attacks.
Arbitrage	$Tr1 \rightarrow Tr2 \rightarrow Tr3 \rightarrow \dots \rightarrow Trn$	Exists a sequence of trades: $Tr1, Tr2, \dots$, and Trn and $Tr1.asset.in == Trn.asset.out$ and $Tr1.asset.out == Tr2.asset.in$ and $Tr2.asset.out == Tr3.asset.in$ and and $Trn - 1.asset.out == Trn.asset.in$	An arbitrageur leverage a sequence of trades ($Tr1, Tr2, \dots$, and Trn) to profit from the price differences crossing multiple exchanges.

a: $\rightarrow$ represents the execution order; b: for better illustration, we do not consider the fee charged by AMM.

Detecting attacks with combination of DeFi actions

- **Demystifying DeFi MEV Activities in Flashbots Bundle**
- <https://dl.acm.org/doi/abs/10.1145/3576915.3616590>
- Detect MEV with **Machine Learning** method

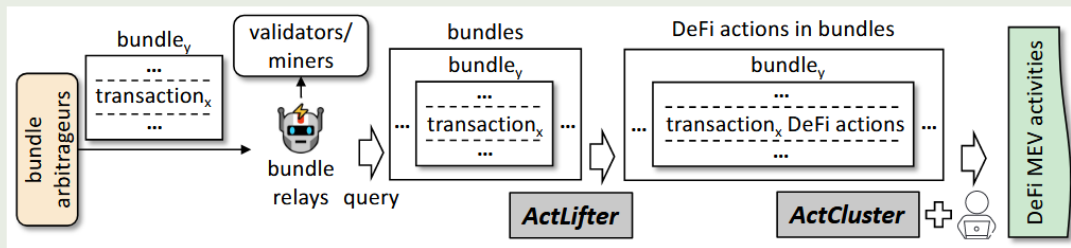


Figure 1: Workflow of discovering DeFi MEV activities in bundles. ACTLIFTER collects the bundles constructed by bundle arbitrageurs and identifies DeFi actions in transactions of each bundle. Inspecting DeFi actions in each bundle, ACTCLUSTER facilitates us to discover DeFi MEV activities.

Workflow with ActLifter and ActCluster

- **ActLifter**: identifying DeFi actions in transactions in Flashbot bundle

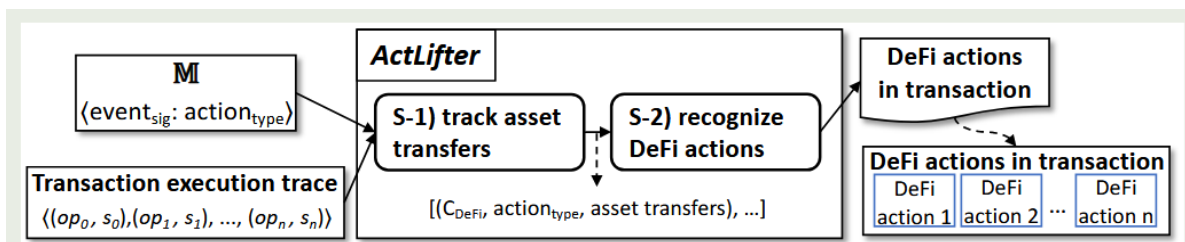


Figure 2: Overview of ACTLIFTER.

- The inputs of ActLifter are **transaction execution trace** and the **mapping M** between the signature of events and the corresponding type of DeFi actions. (manually built by leveraging the event information from developers)
- **S-1**: Identify the specific information of an event (*xx address call xx DeFi contract, did xx action, of xx asset*)
- **S-2**: Get DeFi action type of each event according to the following table

Table 3: Asset transfer patterns of ten DeFi actions

DeFi action type	Asset transfer patterns
Swap	$\frac{Asset_1.Transfer(_, C_{DeFi}, x_1) \wedge Asset_2.Transfer(C_{DeFi}, _, x_2)}{C_{DeFi}.Swap(x_1 : Asset_1, x_2 : Asset_2)}$
AddLiquidity	$\frac{Asset_1.Transfer(_, C_{DeFi}, x_1) \wedge Asset_2.Transfer(_, C_{DeFi}, x_2) \wedge \dots \wedge Asset_n.Transfer(_, C_{DeFi}, x_n)}{C_{DeFi}.AddLiquidity(x_1 : Asset_1, x_2 : Asset_2, \dots, x_n : Asset_n)}$
RemoveLiquidity	$\frac{Asset_1.Transfer(C_{DeFi}, _, x_1) \wedge Asset_2.Transfer(C_{DeFi}, _, x_2) \wedge \dots \wedge Asset_n.Transfer(C_{DeFi}, _, x_n)}{C_{DeFi}.RemoveLiquidity(x_1 : Asset_1, x_2 : Asset_2, \dots, x_n : Asset_n)}$
Leverage	$\frac{Asset_1.Transfer(C_{DeFi}, _, x_1)}{C_{DeFi}.Leverage(x_1 : Asset_1)}$
Borrowing	$\frac{Asset_1.Transfer(C_{DeFi}, _, x_1)}{C_{DeFi}.Borrowing(x_1 : Asset_1)}$
Liquidation	$\frac{Asset_1.Transfer(_, C_{DeFi}, x_1) \wedge Asset_2.Transfer(C_{DeFi}, _, x_2)}{C_{DeFi}.Liquidation(x_1 : Asset_1, x_2 : Asset_2)}$
NFT-Minting	$\frac{Asset_{C_{DeFi}}^{721}.Minting(_, _, x_1)}{C_{DeFi}.NFT-Minting(tokenId_{x_1} : Asset_{C_{DeFi}})}$
NFT-Burning	$\frac{Asset_{C_{DeFi}}^{721}.Burning(_, _, x_1)}{C_{DeFi}.NFT-Burning(tokenId_{x_1} : Asset_{C_{DeFi}})}$
Airdrop	$\frac{Asset_1.Transfer(C_{DeFi}, _, x_1)}{C_{DeFi}.Airdrop(x_1 : Asset_1)}$
Rebasing	$\frac{-}{C_{DeFi}.Rebasing()}$

DeFi Actions defined in this paper

- ActLifter recognizes the corresponding DeFi action according to the asset transfer patterns
- **ActCluster**: discovering unknown DeFi MEV activities in bundles by analyzing the semantic features involved in the sequences of DeFi actions identified by ActLifter.
 1. Cluster bundles with similar activities, and **manually** discover new DeFi MEV activities.
 2. Employ **Representation Learning** to automatically extract distinguishable features from raw bundles.

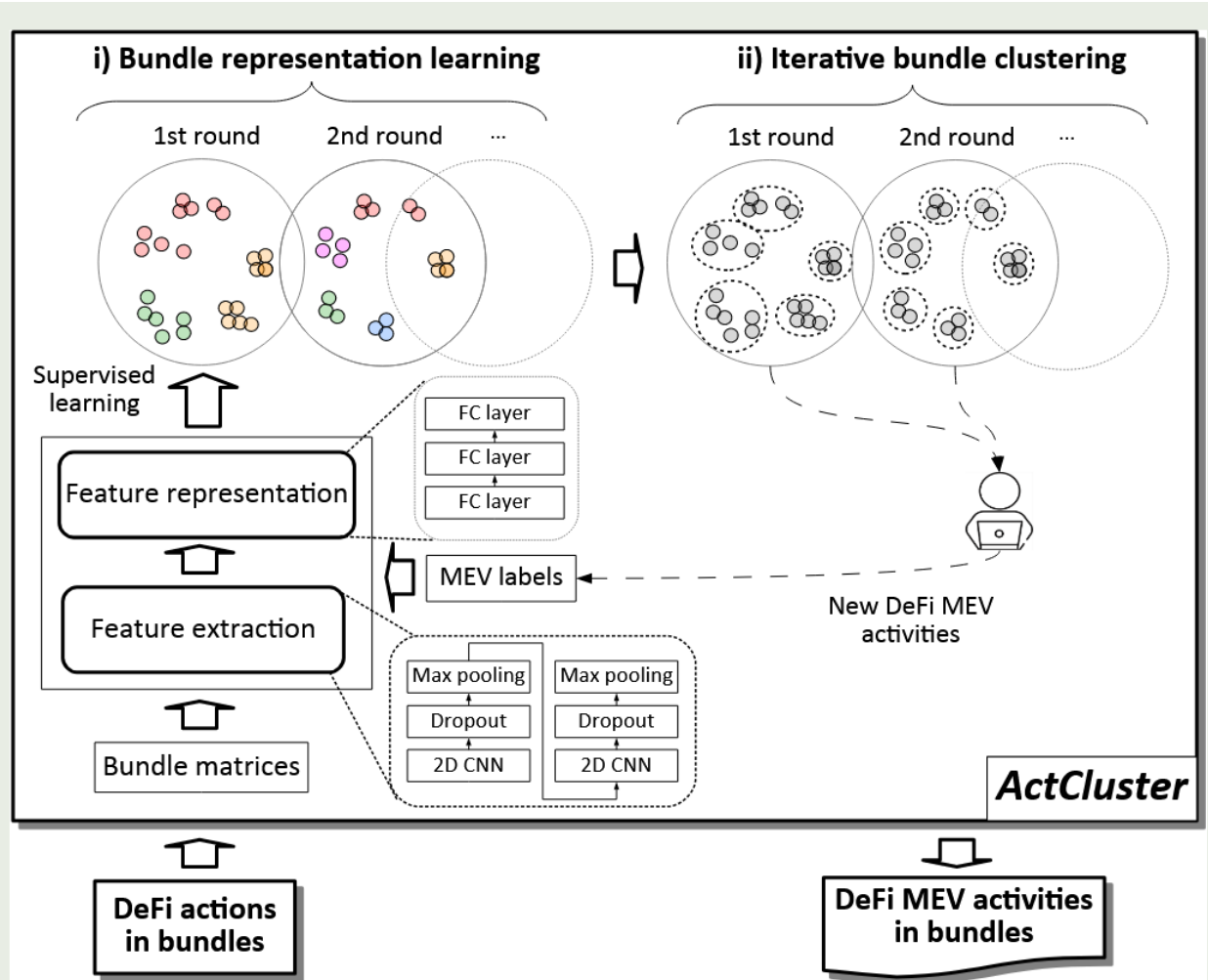
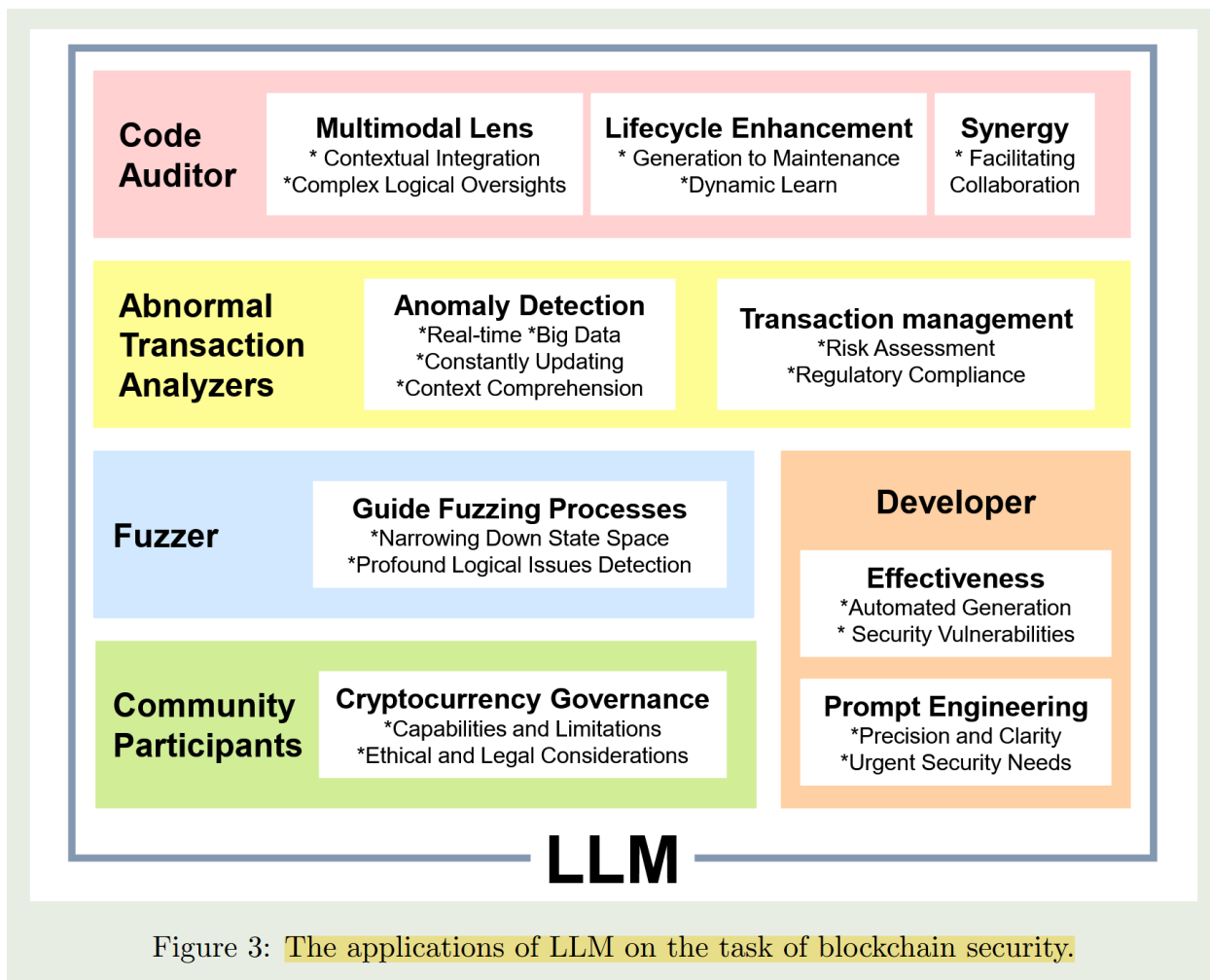


Figure 4: Overview of ACTCluster

- Feature extraction is implemented using stacked blocks consisting of a [2D CNN layer](#), a [dropout layer](#), and a [max pooling layer](#).

Usage of LLM in Blockchain System

- Works on LLM for MEV detection not found at the writing time, but there're many other usages.
- [Large Language Models for Blockchain Security: A Systematic Literature Review](#)
- <https://arxiv.org/abs/2403.14280>



How people use LLM in Blockchain

Table 2: Table of tools and models for code auditor

Research	Functionality	LLM(s)	Publication
SMARTINV [29]	Model finetuning approach	Alpaca etc.	SP,2024
GPTScan [13]	Logic vulnerability detection	GPT-3.5 etc.	ICSE,2024
David et al. [30]	LLM audit feasibility	GPT-4 etc.	arXiv,2023
Karanjai et al. [31]	LLM code evaluation	ChatGPT etc.	BRAINS,2023
ContractArmor [32]	Rule-based code analysis	ChatGPT	EUSPN,2024
Ortu et al. [33]	Contract automatic repair	ChatGPT etc.	arXiv,2023
Jain et al. [41]	Contract automatic repair	GPT-3.5 etc.	ICI,2023
ASSBert [34]	Contract vulnerability detection	BERT	JISA,2023
PSCVFinder [35]	Contract vulnerability detection	CodeT5	ISSRE,2023
Chen et al. [40]	Contract vulnerability detection	ChatGPT etc.	arXiv,2023
LLM4Vuln [36]	Vulnerability reasoning enhancement	GPT-4 etc.	arXiv,2024
TrustLLMf [37]	Smart contracts audit	CodeLlama-13b	arXiv,2024
AuditGPT [38]	ERC token audit	ChatGPT etc.	arXiv,2024
PropertyGPT [39]	Formal verification automation	GPT-4	arXiv,2024
SolGPT [42]	Contract vulnerability detection	GPT-2	ICA3PP,2023

Related works

- **SmartInv: Multimodal Learning for Smart Contract Invariant Inference (IEEE SP 2024, CCF A)**
- <https://ieeexplore.ieee.org/document/10646885>
- A new prompting approach to foundation models, **Tier of Thought (ToT)**, to finetune and elicit their reasoning by following the thought process of human analyzers.

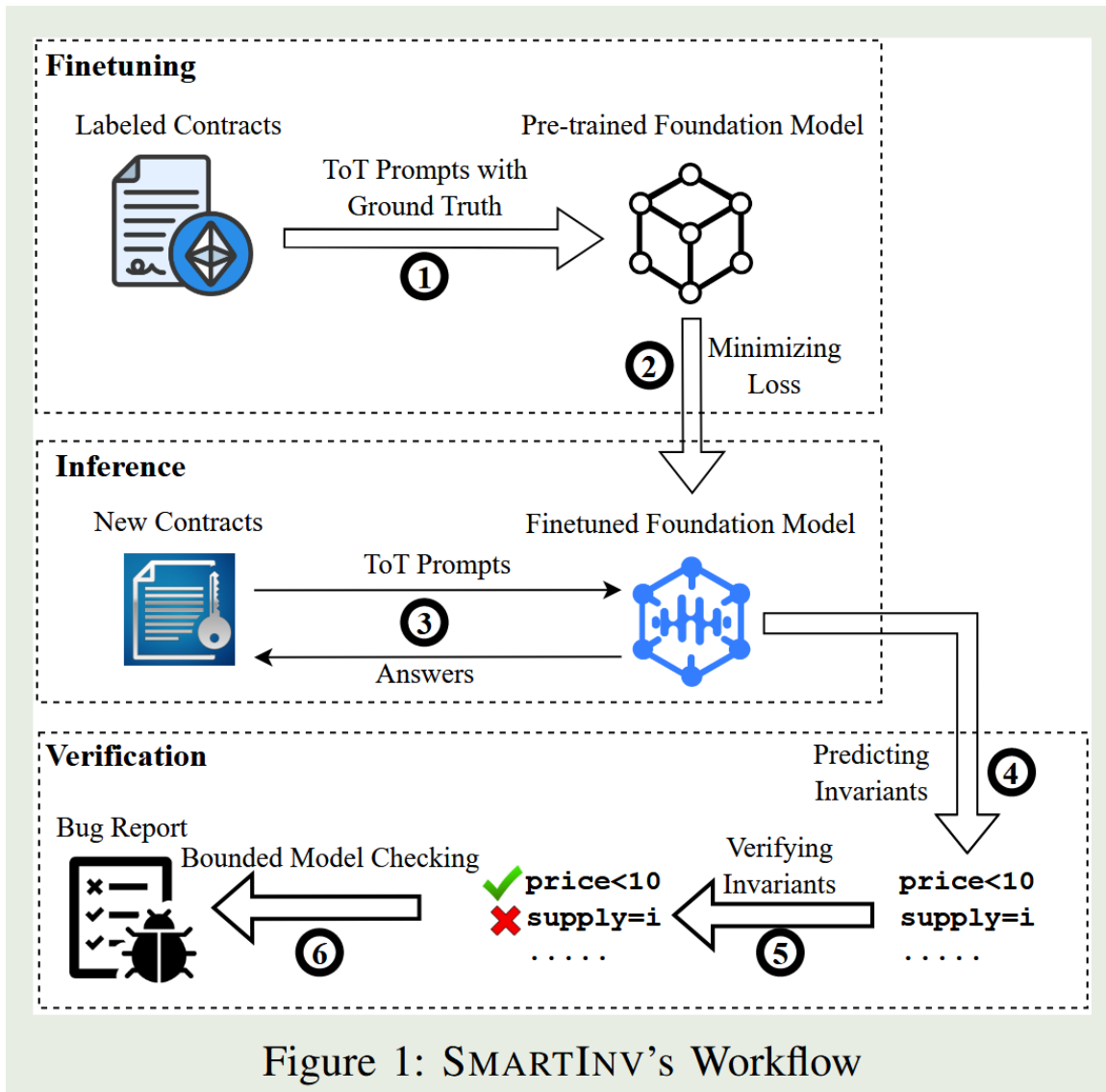


Figure 1: SMARTINV's Workflow

- Increasingly complex thoughts to reason from contract source files to correct answers
- The key of ToT prompting is to **generate one thought at a time**

```

1 contract trainingExample {
2   /* totalSupply and balances should be same before and after
   transfer */
3   uint totalSupply, tokens;
4   mapping(address => uint) balances;
5
6   function transfer(address to) external {
7     balances[to] = balances[to].add(tokens);
8     balances[msg.sender] = balances[msg.sender].sub(tokens);
9   }
10
11  /* only contract owner should invoke tokenIncrease */
12  function tokenIncrease() external {
13    if (tokens <= 100) {
14      tokens += 1.1 * tokens;
15    }
16    return tokens;
17  }
18  ...
19 }

```

Listing 4: Training example contract

Contract trainingExample {...}
 What's the transaction context of the contract? The transaction context is **token transfer**.
 Given transaction context, what are the critical program points? Critical program points are 7+, 8+, 10+, 12, 17+.
 <End of Text>

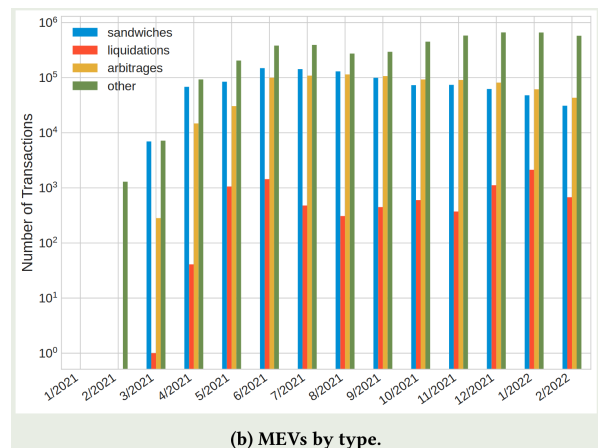
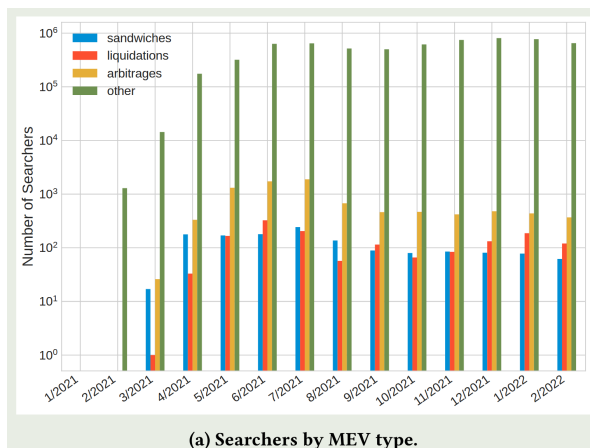
Contract trainingExample {...}
 Given inferred critical program points, what are the invariants? Invariants are [ground truth of "Invariants" label in Table 5].
 Given inferred invariants, what are the critical invariants? Critical invariants are [ground truth of "Critical Invariants" label in Table 5].
 <End of Text>

- Thought = Question + Answer

Is MEV Good or Bad?

Estimation: Is MEV situation severe?

- **A flash(bot) in the pan: measuring maximal extractable value in private pools**
- <https://dl.acm.org/doi/abs/10.1145/3517745.3561448>



MEV Strategy	Extractions	Via Flashbots	Via Flash Loans	Via Both
Sandwiching	1,020,044	485,680 (47.61%)	0 (0%)	0 (0%)
Arbitrage	3,462,678	916,709 (26.47%)	10.155 (0.29%)	1.089 (0.03%)
Liquidation	32,819	9,191 (28.01%)	1.672 (5.09%)	132 (0.4%)
Total	4.515.541	1.411.580 (31.26%)	11.827 (0.26%)	1.221 (0.03%)

Table 1: MEV dataset overview.

- **Miners** gain slightly higher average profits (**0.125 ETH**) on sandwich frontrunning when using Flashbots than without Flashbots (**0.048 ETH**)
- **Searchers** gain lower average profit when using Flashbots (**0.02 ETH**), compared to **0.13 ETH** without Flashbots.
- About **1.58%** MEV transactions are unprofitable.

• **Speculative Multipliers on DeFi: Quantifying On-Chain Leverage Risks**

- https://link.springer.com/chapter/10.1007/978-3-031-18283-9_3
- Leverage Model
- with 1.2B USD of locked value in Alpha Homora (AH), the lenders consistently benefit from a positive APY, while **72.35%** of the leverage taking borrowers suffer from a negative APY, when ignoring the governance token incentivization in AH.

• **Impact and User Perception of Sandwich Attacks in the DeFi Ecosystem**

- Users are not effectively defending themselves against sandwich attacks as of April 2021, indicating a potential gap between users' perceptions of sandwich attacks and their real-world impact.
- Both **insider** and **non-professional** users were unaware of the severity of the sandwich attack

• **Research Note Review and Maximal Extractable Value Blockchain Oracles**

- <https://www.fca.org.uk/publications/research-notes/research-note-review-maximal-extractable-value-and-blockchain-oracles>

- MEV isn't inherently bad or evil – mainly **it just allows many kinds of good things already happening in a healthy DeFi ecosystem**. Liquidity pools stay balanced. And liquidations can become faster – and it keeps those systems more robust. That is what we consider “good MEV”, but of course **there is a dark side too** – which can have a really negative influence...
- **The guys that are doing MEV attacks are not doing anything illegal**. They're also not hacking. They are just exploring the options that are there – in the blockchain. **It's not a hacking attack, it's just a vulnerability**. In blockchain design, blockchain designers never thought that somebody might reorder the transactions.

legitimacy

- **Battle of the Crypto Bots: Automated Transaction Copying in Decentralized Finance**
 - https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4411448
 - Can it ever be legal to copy other people's transactions?
 - The operation to copy or exploit profitable transactions devised by other users submitted publicly via the mempool is generally **not likely to amount to a legal violation** given the distinct way that crypto markets operate. (According to the **Anti-Market Manipulation Rules in U.S.**)
 - Exceptions:
 - the GPS bot is operated by a **validator/block producer** who use their ability to order transactions in order to profit for themselves through strategy copying
 - cases in which the GPS bot copies explicit **private order flow** (i.e. non-public mempool transactions)
 - cases in which the GPS bot copies **criminal or otherwise illicit** transactions where the GPS bot operator is aware of a substantial risk that this is happening.
-
- **Blockchain Transaction Ordering as Market Manipulation**

- https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4187752
- Some forms of MEV extraction including certain common forms of sandwiching entail a **significant risk of amounting to a manipulative practice** in contravention of CFTC Rule 180.1 or SEC Rule 10b-5 (depending on whether the crypto assets at issue end up being classified as commodities or securities).

Mitigation

Private Blockchain (Academic)

- **Themis: Fast, Strong Order-Fairness in Byzantine Consensus**
- <https://dl.acm.org/doi/abs/10.1145/3576915.3616658>
- **Order-Fairness:** let algorithm decide the transaction order

Algorithm FairPropose($\mathcal{L}$)

// Propose a fair ordering for new transactions.
On input a set $\mathcal{L}$ containing the local orderings of $n - f$ nodes:

(1) **Build Dependency Graph**

- Create an empty graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.
- For each non-blank tx, add a vertex tx to $\mathcal{V}$.
- For each tx, tx' $\in \mathcal{V}$, let $k = \text{Weight}_{\mathcal{L}}(\text{tx}, \text{tx}')$ and $k' = \text{Weight}_{\mathcal{L}}(\text{tx}', \text{tx})$. If $k \geq n(1 - \gamma) + \gamma f + 1$ and $k \geq k'$ and $(\text{tx}', \text{tx}) \notin \mathcal{E}$, then add the edge (tx, tx') to $\mathcal{E}$.
- Compute the condensation $\mathcal{G}^*$ and its **topological sorting** S .

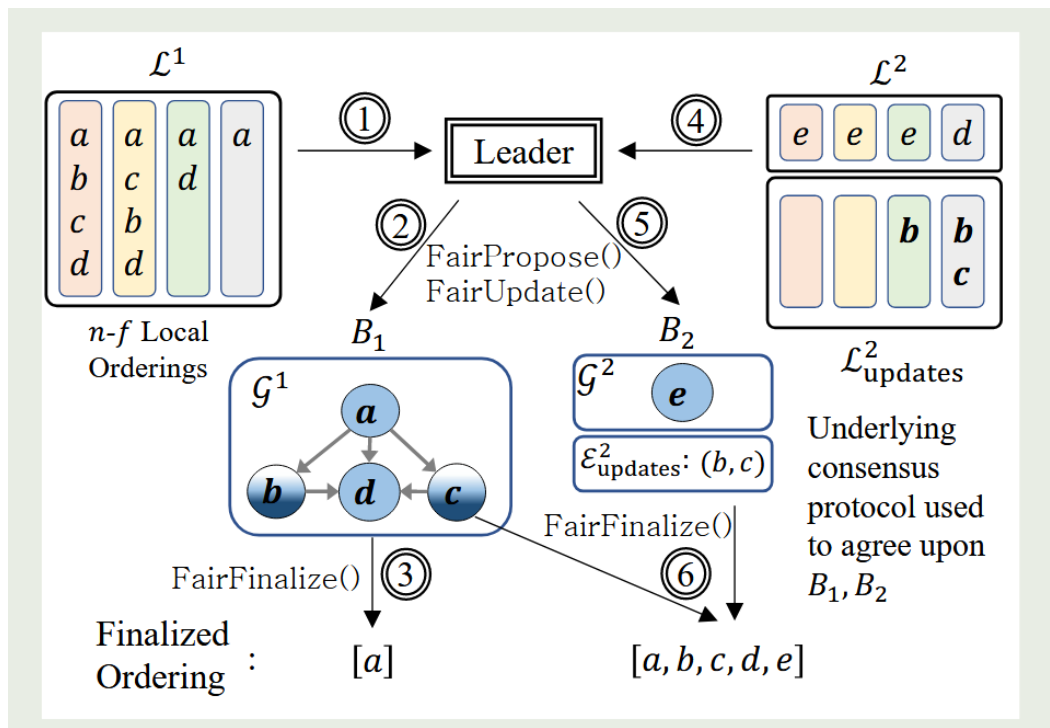
(2) **Output Fair Ordering**

- Let V be the last vertex of S that contains a solid transaction.
- Remove those tx from $\mathcal{G}$, that are in vertices after V in S .
- Output $\mathcal{G}$.

Figure 1: Leader Proposal Algorithm

- Based on $n-f$ replica ordering algorithm. Weight of two transactions is the maximum value k to make tx in the front of tx'.

- Generally, an edge (tx, tx') means adding tx to the order list can bring more fault tolerance to the network than tx' .

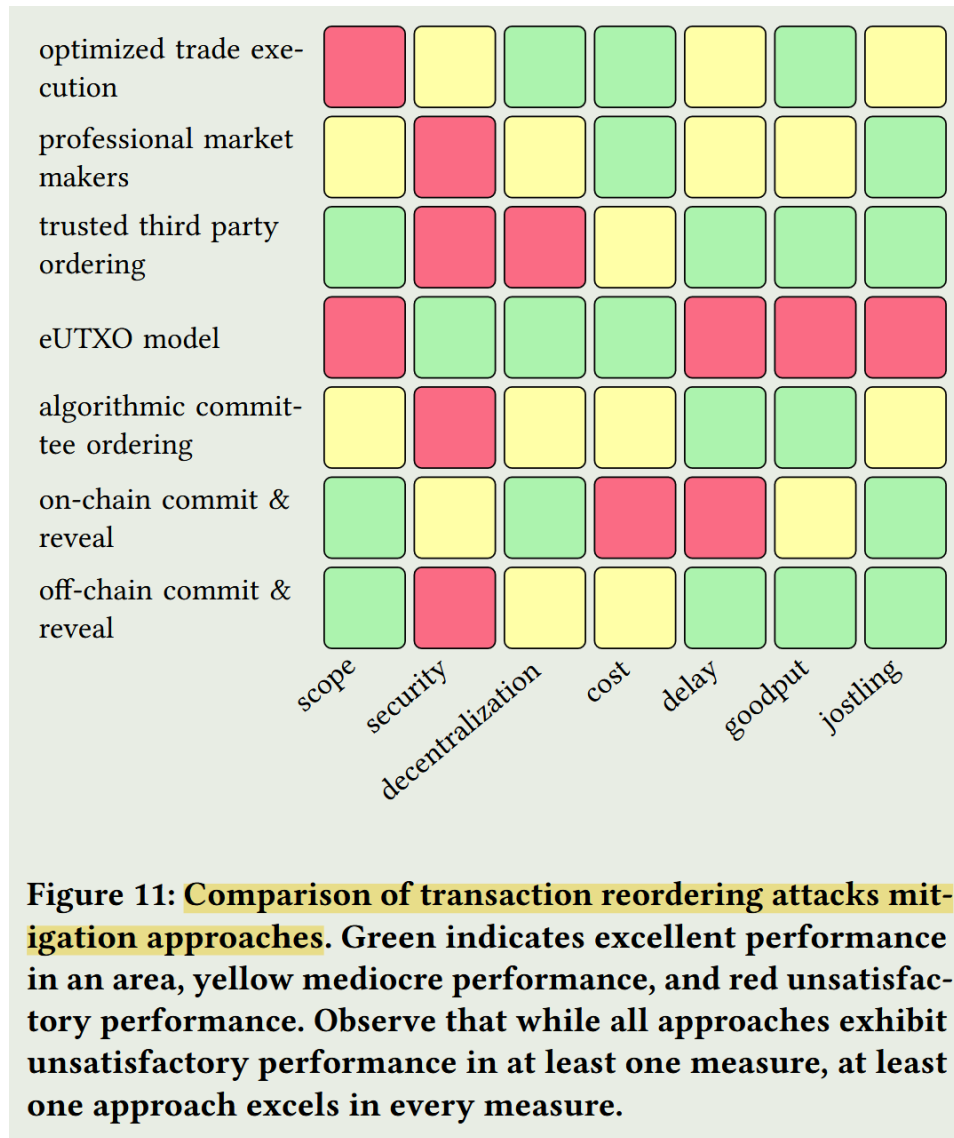


- Ordering list Finality: Transactions in **Hamiltonian cycle** of the Graph are marked as Finalized.

Public Blockchain (Academic)

- **SoK: MEV Countermeasures: Theory and Practice**
- <https://arxiv.org/abs/2212.05111>
 - **Remove the miner's ability to arbitrarily order transactions** and enforce ordering for the transactions.
 - **Limit the visibility of transactions** with cryptographic techniques.
 - Design the Dapps from the bottom-up to **remove the impact of transaction order**.
- **SoK: Preventing Transaction Reordering Manipulations in Decentralized Finance**

- <https://dl.acm.org/doi/abs/10.1145/3558535.3559784>

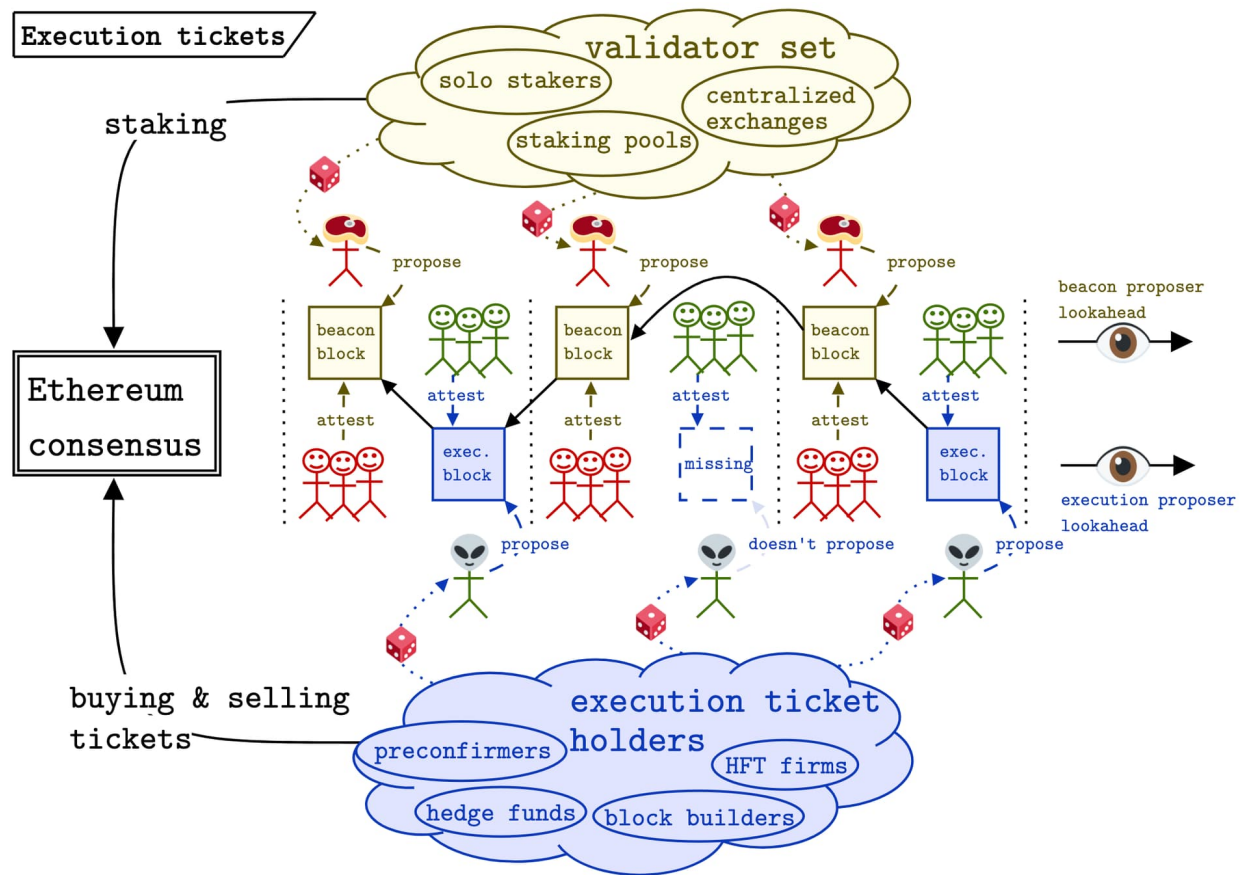


- **Optimized Trade Execution:** limited to particular attacks on specific applications
- **Professional Market Makers:** Re-design DEX with fair-ordering
- **Trusted Third Party Ordering** (Flashbots, OpenMEV)
- **eUTXO Model:** User's coins are not accumulated in their account but instead stored in individual UTXOs linked to their account.

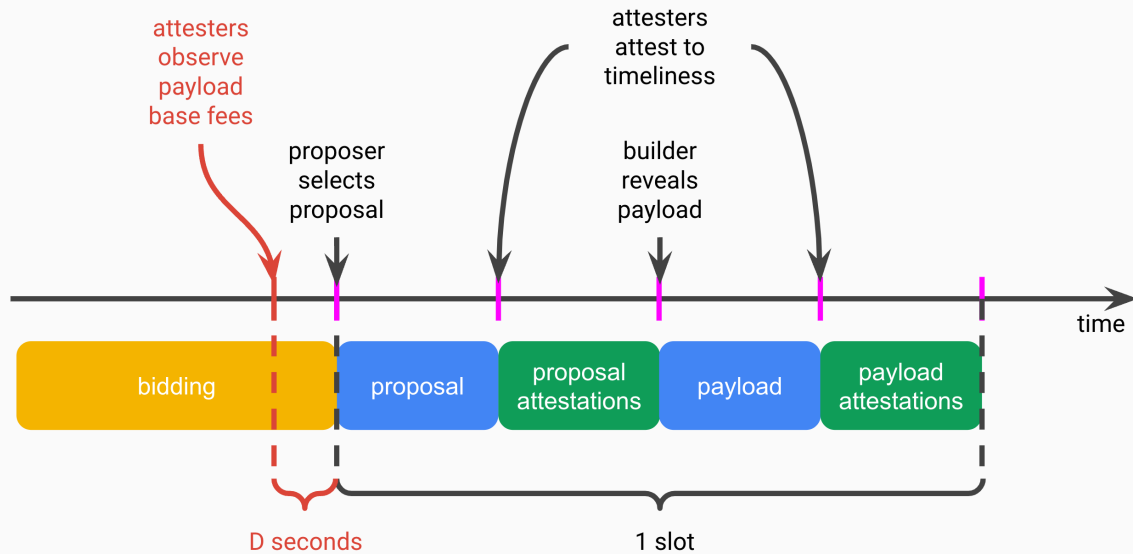
- **Algorithmic Committee Ordering:** in the middle of trusted third party ordering and decentralized ordering,
 - **On-Chain Commit & Reveal:** Users commit to their transactions → transactions are revealed in a later block.
 - **Off-Chain Commit & Reveal:** handled off-chain by a committee
-

Ideas (Industry)

- **Execution Tickets:** <https://ethresear.ch/t/execution-tickets/17944>
- <https://arxiv.org/abs/2404.04262>
- Workflow:
 1. **Proposer** propose a **beacon block** containing an **inclusion list**.
 2. **Attesters** vote on the validity and timeliness of the **beacon block**.
 3. During the **execution round**, a randomly selected **execution proposer** has authorization to propose an **execution block** with **execution ticket**.
 4. The **execution attesters** vote for on the timeliness and validity of the **execution block**.



- Problems:
 - Multi-Block MEV and Centralization Risks
 - Pricing Mechanisms Complexities
- **MEV Burn:** <https://ethresear.ch/t/mev-burn-a-simple-design/15590>



- An add-on to enshrined PBS:
 - The bid from builder are composed of payload base fee + payload tip.
 - The payload base fee is burned, even if the payload is invalid or revealed late.
 - **payload base fee floor:** Honest attesters set their *payload base fee floor* to the top *builder base fee* D seconds (min bid gossip delay) before the end of bidding section.
 - **payload base fee maximization:** Honest proposers select winning bids that maximize the payload base fee.
- Problems:
 - **MEV-Burn fails to burn all potential MEV:** The protocol burn the highest bid in the first D seconds of the auction. And MEV bids increase monotonically over time (The Timing Games). There's a dilemma when setting the parameter D .
 - <https://ethresear.ch/t/timing-games-implications-and-possible-mitigations/17612>
 - **Bypassability:** Unless implemented in protocol, what is the reason to follow this game? There is no credible method to ensure all proposers and builders burn their profit.

-
- **MEV-Smoothing:** <https://notes.ethereum.org/cA3EzpNvRBStk1JFLzW8qg>
 - Key insight: **Pay the payload base fee forward** rather than burning it, and **smooth out the the variability in the earning of all Validators**.
 - **Committee-Driven Smoothing:** The payment is split equally among all committee members and to the proposer, which would currently be splitting it to about 6250 validators
 - Philosophical Consideration: Protocol should not intervene in artificially smoothing rewards, but allow for a free market where individuals and entities decide their own tolerance for variances in earnings. (the potential for centralization)
-
- **Shutterized Beacon Chain:**<https://ethresear.ch/t/shutterized-beacon-chain/12249>
 - **A Commit & Reveal Method**
 - A set of validator compute an encryption key using a DKG protocol (Distributed Key Generation).
 - Users encrypt their transactions with it.
 - The decryption key will be released once the encrypted transactions are on the chain.
 - Problem:
 - How to define the **Correctness** of a block?
 - Execution layer clients need to get the decryption key before execution of transactions, which putting more pressure on the P2P network.
-

Personal Opinions

- Despite the enormous economic benefits of MEVs, whether MEV, an unexpected occurrence, is entirely a good thing that energizes the market, or a combination of both positive (arbitrage) and negative (sandwich) effects, or totally detrimental by increasing the cost of using blockchain for everyone?

- When people are making tremendous profit from MEV, does anyone stopped to ask: **What is the Price?** The price will impact everyone who try to deploy applications on blockchain, they have to consider how to deal with MEV in applications. This is the **Price**, although most developers don't care. The price will affect everyone who tries to use applications on blockchain, they have to consider if their transactions will be MEVed when they try to participate. This is the **Price**, although few people care about it.